

LoRa P2P Communication with STM32 and ESP32

A Practical How-To Guide

Aske Mop Rasmussen
Technical University of Denmark

July 2026

Contents

1	Introduction	3
1.1	Hardware Used	3
1.2	Prerequisites	3
2	Part 1: Setting up the ESP32 Development Environment	4
2.1	Installing the ESP-IDF Extension in VSCode	4
2.2	Configuring the Extension	4
2.3	Creating and Flashing a Test Project	4
2.4	Building and Flashing	5
3	Part 2: Wiring the Ra-01 LoRa Module to the ESP32	6
3.1	Pin Connections	6
3.2	Wiring Notes	6
4	Part 3: ESP32 LoRa TX/RX Implementation	7
4.1	Installing the sx127x Library	7
4.2	Configuring Pin Numbers via Menuconfig	7
4.3	Setting Transmit Power for EU Compliance	8
4.4	Setting Up Two Boards	8
4.5	Configuring TX/RX Mode	8
4.6	Building and Flashing	9
4.7	Verifying P2P Communication	9
5	Part 4: Setting up STM32CubeIDE & STM32CubeMX	11
5.1	Installing STM32CubeIDE & STM32CubeMX Software	11
5.2	Creating the Project in STM32CubeMX	11
5.3	Opening the Project in STM32CubeIDE	12
6	Part 5: Wiring the Ra-01 LoRa Module to the STM32F302	13
6.1	Pin Connections	13
6.2	Wiring Notes	13

7	Part 6: Porting the LoRa Driver to STM32F302	14
7.1	Selecting a Portable LoRa Driver	14
7.2	Setting Transmit Power for EU Compliance	14
7.3	Integrating the Driver	15
7.4	Writing the Receiver Code	15
7.5	Verifying Communication	16
7.6	Adding Transmit Capability	17
7.7	Testing STM32 to ESP32 Transmission	17
8	Part 7: LoRa Parameters and P2P Communication Tests	18
8.1	Understanding LoRa Communication Parameters	18
8.1.1	Spreading Factor (SF)	18
8.1.2	Bandwidth (BW)	18
8.1.3	Coding Rate (CR)	18
8.2	EU 433 MHz Power Limits and Duty Cycle	19
8.3	Test Setup	19
8.4	Test 1: ESP32 to ESP32	19
8.5	Test 2: ESP32 to STM32F302	20
8.6	Throughput	20
8.7	Notes for Further Testing	20

1 Introduction

This guide documents the practical implementation of LoRa Point-to-Point (P2P) communication between an ESP32 and an STM32F302 microcontroller using the Ra-01 (SX1278) module via SPI. The guide is intended to serve as a reusable reference for future students.

1.1 Hardware Used

- ESP32 DevKit (USB-C)
- STM32F302 Nucleo-64
- 2x Ra-01 (SX1278) LoRa modules
- 2x 433 MHz antennas
- Breadboard, jumper wires, USB cables

1.2 Prerequisites

- Visual Studio Code installed
- Basic familiarity with embedded C
- macOS, Windows, or Linux machine

Note: This guide was written and tested on macOS. File paths and serial port names will differ on Windows and Linux. Refer to the official ESP-IDF documentation for platform-specific instructions.

2 Part 1: Setting up the ESP32 Development Environment

2.1 Installing the ESP-IDF Extension in VSCode

1. Install VSCode from <https://code.visualstudio.com>
2. Install the ESP-IDF extension from the VSCode marketplace: <https://marketplace.visualstudio.com/items?itemName=espressif.esp-idf-extension>
3. If on Mac/Linux, ensure prerequisites are installed: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/get-started/index.html#installation>

2.2 Configuring the Extension

1. Open the ESP-IDF welcome page and click **Configure ESP-IDF Extension**
2. Select **Use Existing Setup** if ESP-IDF was already installed via EIM
3. Click **Search ESP-IDF in system**
4. If the path is not found automatically, enter it manually. On macOS:

```
/Users/<username>/.espressif/v6.0.2/esp-idf
```

5. Once the path is valid, click **Install**
6. You should see *"All settings have been configured"*

2.3 Creating and Flashing a Test Project

1. Plug in your ESP32 before creating the project so the serial port is detected
2. Click **New Project** and fill in:
 - Project Name: `esp32-lora-ra01`
 - IDF Target: `esp32`
 - Board: `Custom board`
 - Serial Port: `/dev/cu.usbserial-XXXX` (macOS)
3. Click **Choose Template** and select `hello_world` under `get-started`
4. Click **Create project using template hello_world**

2.4 Building and Flashing

Open a terminal and navigate to the project folder:

```
cd ~/esp32-lora-ra01
```

Source the ESP-IDF environment (required every new terminal session):

```
source ~/.espressif/v6.0.2/esp-idf/export.sh
```

If a build folder exists from a previous failed attempt, remove it:

```
rm -rf build
```

Build, flash, and monitor:

```
idf.py build
idf.py flash
idf.py monitor
```

You should see the following output confirming the board is working:

```
Hello world!
This is esp32 chip with 2 CPU core(s), WiFi/BT/BLE, silicon
  revision v3.1
Restarting in 10 seconds...
```

To exit the monitor press **Ctrl+]**.

3 Part 2: Wiring the Ra-01 LoRa Module to the ESP32

The Ra-01 communicates with the ESP32 via SPI (Serial Peripheral Interface). The module runs on 3.3V. Do **not** connect it to 5V, as this will permanently damage it.

3.1 Pin Connections

Ra-01 Pin	Board Label	GPIO	Function
3.3V	3V3	–	Power
GND	GND	–	Ground
NSS	D5	GPIO 5	SPI Chip Select
SCK	D18	GPIO 18	SPI Clock
MISO	D19	GPIO 19	SPI Data Out
MOSI	D23	GPIO 23	SPI Data In
RST	D4	GPIO 4	Reset
DIO0	D2	GPIO 2	Interrupt (TX/RX done)

Note: Highlighted rows (GPIO 18, 19, 23) are the ESP32's dedicated hardware SPI pins and must not be changed.

3.2 Wiring Notes

- GPIO 18, 19, and 23 are the ESP32's default hardware SPI pins and must be used for SCK, MISO, and MOSI respectively
- GPIO 5 is the default SPI chip select pin
- GPIO 4 and GPIO 2 were chosen for RST and DIO0 as they were available on the right side of the DevKit when mounted on a breadboard
- Place the ESP32 straddling the centre gap of the breadboard so pins are accessible on both sides

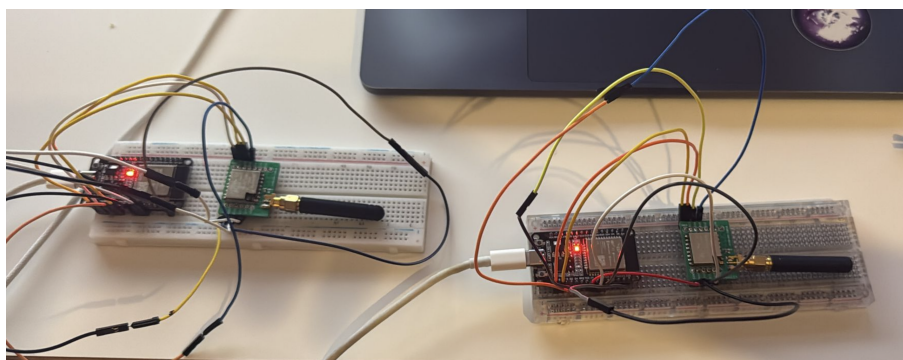


Figure 1: ESP32 wired to the Ra-01 LoRa Module

4 Part 3: ESP32 LoRa TX/RX Implementation

4.1 Installing the sx127x Library

Rather than writing a driver from scratch, this guide uses the `esp-idf-sx127x` library by nopnop2002, which provides a ready-made ESP-IDF compatible LoRa driver with a basic transmit/receive example.

Clone the repository:

```
cd ~
git clone https://github.com/nopnop2002/esp-idf-sx127x
cd esp-idf-sx127x/basic/
```

4.2 Configuring Pin Numbers via Menuconfig

Source the ESP-IDF environment if not already done:

```
source ~/.espressif/v6.0.2/esp-idf/export.sh
```

Open the configuration menu:

```
idf.py menuconfig
```

Navigate to **LoRa Configuration** and set the following:

- Frequency: 433MHz (default, correct)
- MISO GPIO: 19 (default, correct)
- SCK GPIO: 18 (default, correct)
- MOSI GPIO: 23 (default, correct)
- NSS GPIO: change to 5
- RST GPIO: change to 4

Press **S** to save and **Q** to quit.

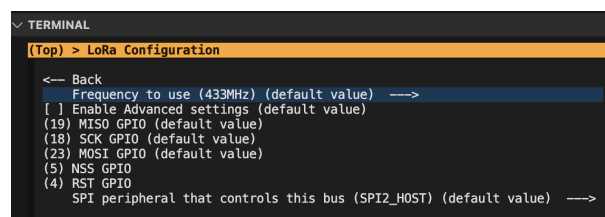


Figure 2: Pin Configuration

4.3 Setting Transmit Power for EU Compliance

The EU 433 MHz ISM band limits transmit power to 10 dBm (10 mW) under EN 300 220. The library's default in `lora_init()` is 17 dBm, which exceeds this limit and must be overridden in your own project code.

In `main/main.c`, add the following line immediately after the `lora_init()` check, before the frequency configuration block:

```
if (lora_init() == 0) {
    ESP_LOGE(pcTaskGetName(NULL), "Does not recognize the module");
    while(1) {
        vTaskDelay(1);
    }
}

lora_set_tx_power(10); // EU 433MHz ISM band limit: 10dBm (10mW)
                        per EN 300 220

#ifdef CONFIG_433MHZ
```

Unlike the STM32 driver (see Part 6), this library's power scale corresponds directly to dBm (2-17), so no lookup table adjustment is needed.

4.4 Setting Up Two Boards

This project requires two identical hardware setups: one acting as the transmitter and one as the receiver. Both boards should be wired identically as described in Part 2.

To avoid confusion when flashing, connect only one board at a time. Flash the transmitter first, then unplug it, connect the receiver and flash that. Once both are flashed, you can connect both simultaneously and monitor them in separate terminal windows.

To identify which port corresponds to which board, run:

```
ls /dev/cu.*
```

Connect one board at a time and note which port appears. On macOS, ports typically look like `/dev/cu.usbserial-XXXX`.

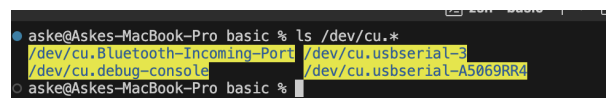


Figure 3: Identify Boards

4.5 Configuring TX/RX Mode

The `basic` example supports both transmitter and receiver modes, configured via menuconfig. To set the mode:

1. Run `idf.py menuconfig`
2. Navigate to **Application Configuration** → **Communication polarity**

3. Select **As the sender** or **As the receiver**
4. Press **S** to save and **Q** to quit

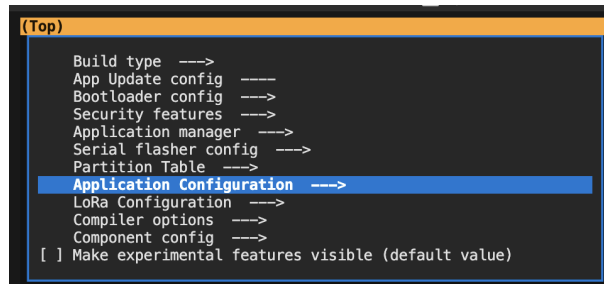


Figure 4: Application Configuration

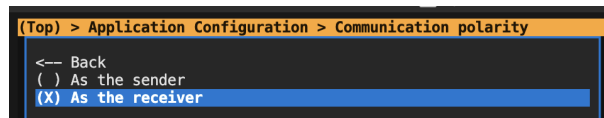


Figure 5: Set as receiver

4.6 Building and Flashing

Build and flash as before:

```
idf.py build
idf.py flash
```

If multiple boards are connected, specify the port explicitly:

```
idf.py -p /dev/cu.usbserial-XXXX flash
```

HINT: Check Section 3.3 to find out the specific port.

4.7 Verifying P2P Communication

With the transmitter flashed on one ESP32 and the receiver on the other, open two terminal windows and run the monitor on each:

```
idf.py -p /dev/cu.usbserial-XXXX monitor
```

For example,

```
Terminal 1: idf.py -p /dev/cu.usbserial-3
Terminal 2: idf.py -p /dev/cu.usbserial-A5069RR4
```

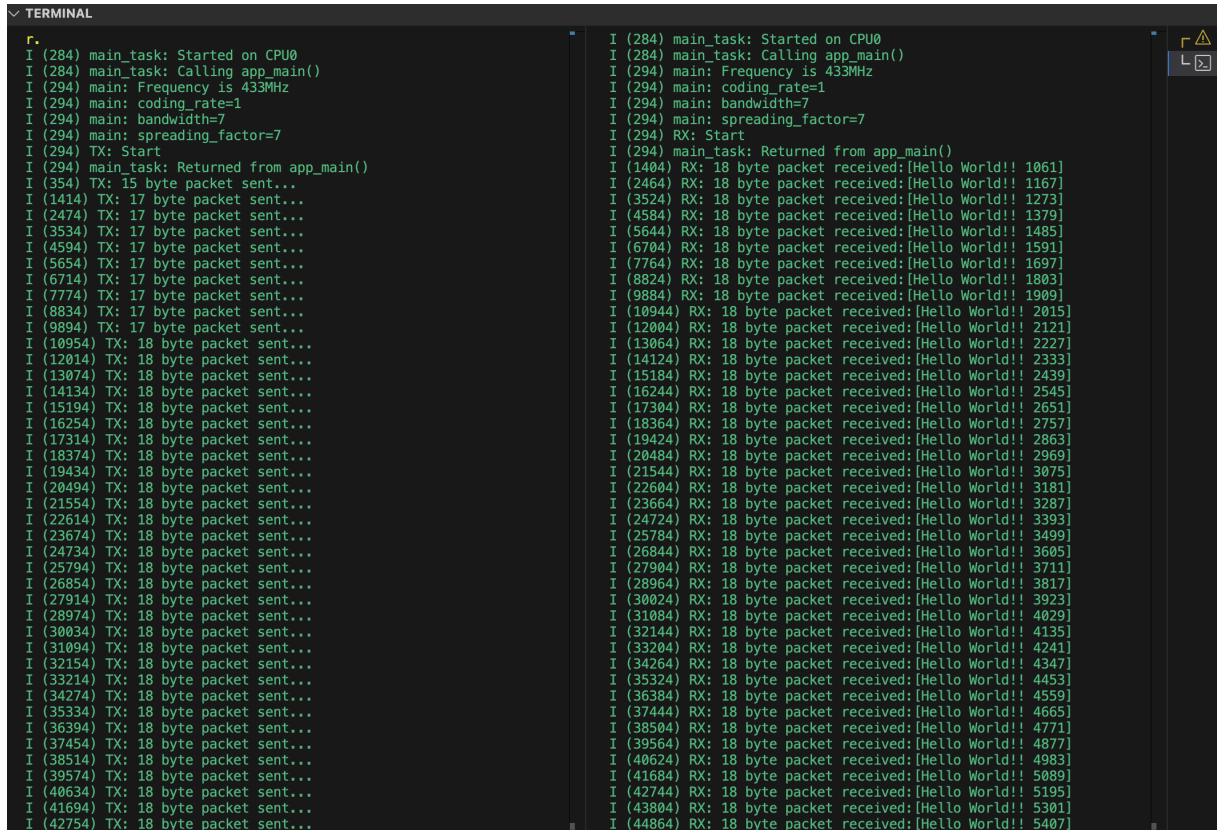
The transmitter should output:

```
I (294) main: Frequency is 433MHz
I (294) TX: Start
I (354) TX: 15 byte packet sent...
I (1414) TX: 17 byte packet sent...
```

The receiver should output:

```
I (294) main: Frequency is 433MHz
I (294) RX: Start
I (1404) RX: 18 byte packet received:[Hello World!! 1061]
I (2464) RX: 18 byte packet received:[Hello World!! 1167]
```

This confirms successful LoRa P2P communication between the two ESP32 boards via the Ra-01 modules.



```

r.
I (284) main_task: Started on CPU0
I (284) main_task: Calling app_main()
I (294) main: Frequency is 433MHz
I (294) main: coding_rate=1
I (294) main: bandwidth=7
I (294) main: spreading_factor=7
I (294) TX: Start
I (294) main_task: Returned from app_main()
I (354) TX: 15 byte packet sent...
I (1414) TX: 17 byte packet sent...
I (2474) TX: 17 byte packet sent...
I (3534) TX: 17 byte packet sent...
I (4594) TX: 17 byte packet sent...
I (5654) TX: 17 byte packet sent...
I (6714) TX: 17 byte packet sent...
I (7774) TX: 17 byte packet sent...
I (8834) TX: 17 byte packet sent...
I (9894) TX: 17 byte packet sent...
I (10954) TX: 18 byte packet sent...
I (12014) TX: 18 byte packet sent...
I (13074) TX: 18 byte packet sent...
I (14134) TX: 18 byte packet sent...
I (15194) TX: 18 byte packet sent...
I (16254) TX: 18 byte packet sent...
I (17314) TX: 18 byte packet sent...
I (18374) TX: 18 byte packet sent...
I (19434) TX: 18 byte packet sent...
I (20494) TX: 18 byte packet sent...
I (21554) TX: 18 byte packet sent...
I (22614) TX: 18 byte packet sent...
I (23674) TX: 18 byte packet sent...
I (24734) TX: 18 byte packet sent...
I (25794) TX: 18 byte packet sent...
I (26854) TX: 18 byte packet sent...
I (27914) TX: 18 byte packet sent...
I (28974) TX: 18 byte packet sent...
I (30034) TX: 18 byte packet sent...
I (31094) TX: 18 byte packet sent...
I (32154) TX: 18 byte packet sent...
I (33214) TX: 18 byte packet sent...
I (34274) TX: 18 byte packet sent...
I (35334) TX: 18 byte packet sent...
I (36394) TX: 18 byte packet sent...
I (37454) TX: 18 byte packet sent...
I (38514) TX: 18 byte packet sent...
I (39574) TX: 18 byte packet sent...
I (40634) TX: 18 byte packet sent...
I (41694) TX: 18 byte packet sent...
I (42754) TX: 18 byte packet sent...

I (284) main_task: Started on CPU0
I (284) main_task: Calling app_main()
I (294) main: Frequency is 433MHz
I (294) main: coding_rate=1
I (294) main: bandwidth=7
I (294) main: spreading_factor=7
I (294) RX: Start
I (294) main_task: Returned from app_main()
I (1404) RX: 18 byte packet received:[Hello World!! 1061]
I (2464) RX: 18 byte packet received:[Hello World!! 1167]
I (3524) RX: 18 byte packet received:[Hello World!! 1273]
I (4584) RX: 18 byte packet received:[Hello World!! 1379]
I (5644) RX: 18 byte packet received:[Hello World!! 1485]
I (6704) RX: 18 byte packet received:[Hello World!! 1591]
I (7764) RX: 18 byte packet received:[Hello World!! 1697]
I (8824) RX: 18 byte packet received:[Hello World!! 1803]
I (9884) RX: 18 byte packet received:[Hello World!! 1909]
I (10944) RX: 18 byte packet received:[Hello World!! 2015]
I (12004) RX: 18 byte packet received:[Hello World!! 2121]
I (13064) RX: 18 byte packet received:[Hello World!! 2227]
I (14124) RX: 18 byte packet received:[Hello World!! 2333]
I (15184) RX: 18 byte packet received:[Hello World!! 2439]
I (16244) RX: 18 byte packet received:[Hello World!! 2545]
I (17304) RX: 18 byte packet received:[Hello World!! 2651]
I (18364) RX: 18 byte packet received:[Hello World!! 2757]
I (19424) RX: 18 byte packet received:[Hello World!! 2863]
I (20484) RX: 18 byte packet received:[Hello World!! 2969]
I (21544) RX: 18 byte packet received:[Hello World!! 3075]
I (22604) RX: 18 byte packet received:[Hello World!! 3181]
I (23664) RX: 18 byte packet received:[Hello World!! 3287]
I (24724) RX: 18 byte packet received:[Hello World!! 3393]
I (25784) RX: 18 byte packet received:[Hello World!! 3499]
I (26844) RX: 18 byte packet received:[Hello World!! 3605]
I (27904) RX: 18 byte packet received:[Hello World!! 3711]
I (28964) RX: 18 byte packet received:[Hello World!! 3817]
I (30024) RX: 18 byte packet received:[Hello World!! 3923]
I (31084) RX: 18 byte packet received:[Hello World!! 4029]
I (32144) RX: 18 byte packet received:[Hello World!! 4135]
I (33204) RX: 18 byte packet received:[Hello World!! 4241]
I (34264) RX: 18 byte packet received:[Hello World!! 4347]
I (35324) RX: 18 byte packet received:[Hello World!! 4453]
I (36384) RX: 18 byte packet received:[Hello World!! 4559]
I (37444) RX: 18 byte packet received:[Hello World!! 4665]
I (38504) RX: 18 byte packet received:[Hello World!! 4771]
I (39564) RX: 18 byte packet received:[Hello World!! 4877]
I (40624) RX: 18 byte packet received:[Hello World!! 4983]
I (41684) RX: 18 byte packet received:[Hello World!! 5089]
I (42744) RX: 18 byte packet received:[Hello World!! 5195]
I (43804) RX: 18 byte packet received:[Hello World!! 5301]
I (44864) RX: 18 byte packet received:[Hello World!! 5407]
```

Figure 6: TX/RX

5 Part 4: Setting up STM32CubeIDE & STM32CubeMX

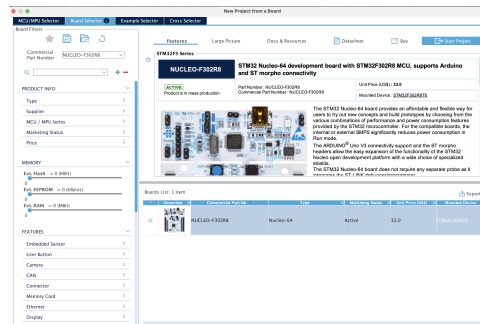
5.1 Installing STM32CubeIDE & STM32CubeMX Software

1. Install STM32CubeIDE from <https://www.st.com/en/development-tools/stm32cubeide.html> (This guide uses the standalone IDE, not the VSCode extension)
2. Install STM32CubeMX from <https://www.st.com/en/development-tools/stm32cubemx.html>

Note: As of STM32CubeIDE 2.0.0+, the CubeMX plugin and .ioc file generation were removed from the IDE. Pin configuration must be done in the standalone STM32CubeMX application, with the generated project then opened in STM32CubeIDE.

5.2 Creating the Project in STM32CubeMX

1. Open STM32CubeMX → **Access to Board Selector** → type F302R8 in Commercial Part Number → select **NUCLEO-F302R8** → **Start Project**



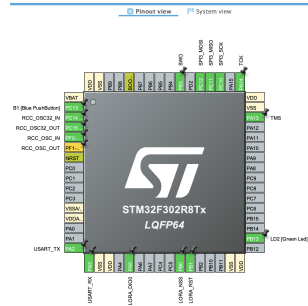
2. Expand **Connectivity** in the left panel → select **SPI3** → set Mode to **Full-Duplex Master**

In the Configuration panel below, under **Parameter Settings** → **Basic Parameters**, set **Data Size** to **8 Bits**. This is required because the LoRa driver communicates in single-byte transactions; if left at a different value, SPI communication with the module will fail.



3. Configure the three additional GPIO pins required by the Ra-01 module:
 - (a) **NSS pin:** select **PB0** → set to **GPIO_Output** → right-click the pin and rename to **LORA_NSS**

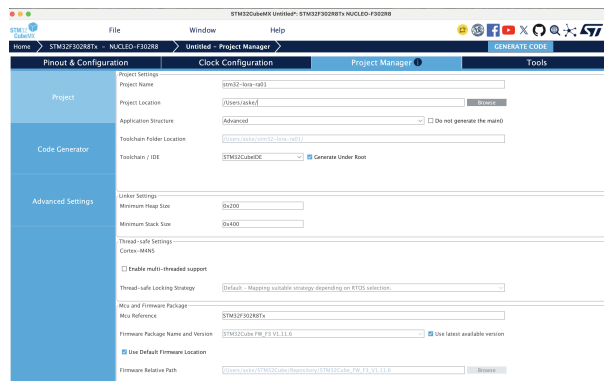
- (b) **RST pin:** select **PB1** → set to **GPIO_Output** → right-click the pin and rename to **LORA_RST**
- (c) **DIO0 interrupt pin:** select **PA5** → set to **GPIO_Input** → right-click the pin and rename to **LORA_DIO0**



4. Navigate to **Project Manager** at the top and configure:

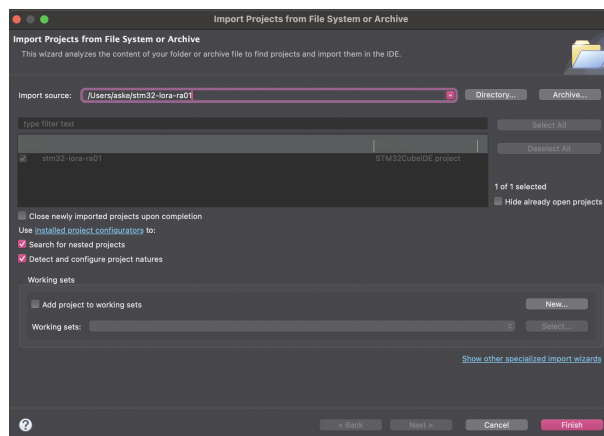
- Project Name: **stm32-lora-ra01**
- Project Location: **/Users/(NAME)/**
- Toolchain / IDE: **STM32CubeIDE**

Then click **GENERATE CODE**.



5.3 Opening the Project in STM32CubeIDE

1. In STM32CubeIDE, go to **File → Open Projects from File System** → browse to **/Users/(NAME)/stm32-lora-ra01** and import it



2. Confirm the pin configuration was generated correctly by opening `Core/Inc/main.h` and checking for the following defines:

```
#define LORA_DIO0_Pin GPIO_PIN_5
#define LORA_DIO0_GPIO_Port GPIOA
#define LORA_NSS_Pin GPIO_PIN_0
#define LORA_NSS_GPIO_Port GPIOB
#define LORA_RST_Pin GPIO_PIN_1
#define LORA_RST_GPIO_Port GPIOB
```

6 Part 5: Wiring the Ra-01 LoRa Module to the STM32F302

The same Ra-01 module setup used with the ESP32 (Part 2) is reused here, rewired to the STM32F302's Morpho headers (CN7/CN10) using the pins configured in Part 4.

6.1 Pin Connections

Ra-01 Pin	STM32 Pin	Function
3.3V	3V3	Power
GND	GND	Ground
NSS	PB0	SPI Chip Select
SCK	PC10	SPI Clock
MISO	PC11	SPI Data Out
MOSI	PC12	SPI Data In
RST	PB1	Reset
DIO0	PA5	Interrupt (TX/RX done)

Note: Highlighted rows are the STM32's SPI3 pins configured in Part 4.

6.2 Wiring Notes

- These pins are located on the outer **Morpho headers** (CN7 and CN10), not the inner Arduino-style D-numbered headers. The exact position varies by board revision. Layout diagrams can be found at https://os.mbed.com/media/uploads/bcostm/nucleo_f302r8_morpho_left_2017_3_23.png and https://os.mbed.com/media/uploads/bcostm/nucleo_f302r8_morpho_right_2017_3_23.png
- The Nucleo board's header pins allow jumper wires to be plugged in directly, without needing a breadboard for the STM32 side.

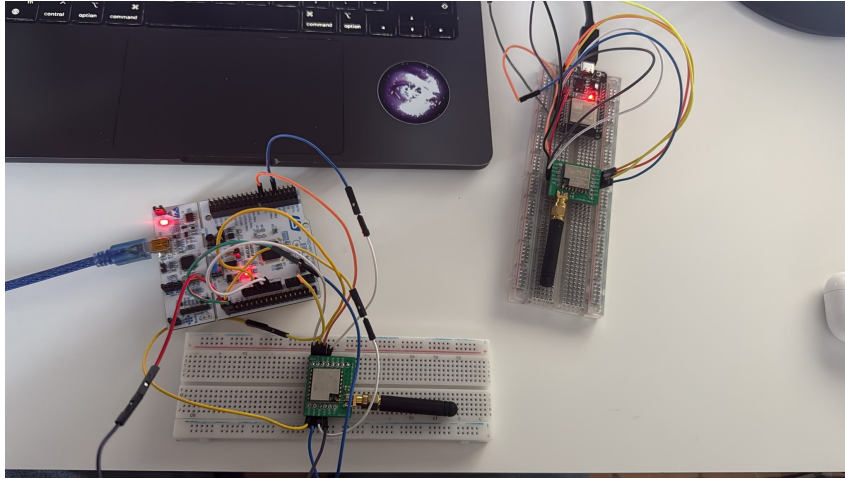


Figure 7: STM32F302 wired to the Ra-01 LoRa Module

7 Part 6: Porting the LoRa Driver to STM32F302

7.1 Selecting a Portable LoRa Driver

The `nopnop2002` library used for the ESP32 (Part 3) is written specifically for the ESP-IDF framework and cannot be used on STM32, which uses a different framework (STM32 HAL). Rather than writing an SPI driver from scratch, this guide uses the `SX1278` driver by Wojciech Domski, which is written in plain C using STM32 HAL calls directly and is designed to be portable across STM32 projects.

Clone the driver and its accompanying example:

```
cd ~
git clone https://github.com/wdomski/SX1278.git
git clone https://github.com/wdomski/SX1278-example.git
```

7.2 Setting Transmit Power for EU Compliance

The EU 433 MHz ISM band limits transmit power to 10 dBm (10 mW) under EN 300 220. This driver only exposes four discrete power presets:

```
#define SX1278_POWER_20DBM      0
#define SX1278_POWER_17DBM      1
#define SX1278_POWER_14DBM      2
#define SX1278_POWER_11DBM      3

static const uint8_t SX1278_Power[4] = { 0xFF, //20dbm
                                           0xFC, //17dbm
                                           0xF9, //14dbm
                                           0xF6, //11dbm
                                           };
```

None of these match 10 dBm exactly. Each step in the table changes the register value by 3 (corresponding to 3 dB), following the pattern `labelled_dBm = (register & 0x0F) + 5`. To add a precise 10 dBm option, edit `Core/Inc/SX1278.h` to extend the table:

```

#define SX1278_POWER_20DBM          0
#define SX1278_POWER_17DBM          1
#define SX1278_POWER_14DBM          2
#define SX1278_POWER_11DBM          3
#define SX1278_POWER_10DBM          4

static const uint8_t SX1278_Power[5] = { 0xFF, //20dbm
                                           0xFC, //17dbm
                                           0xF9, //14dbm
                                           0xF6, //11dbm
                                           0xF5, //10dbm
                                           };

```

This new option is used in the `SX1278_init()` call in the next section.

7.3 Integrating the Driver

Copy the driver source files into the STM32 project:

```

cp ~/SX1278/driver/*.c ~/stm32-lora-ra01/Core/Src/
cp ~/SX1278/driver/*.h ~/stm32-lora-ra01/Core/Inc/

```

Since this project uses the simplified CubeIDE structure (all peripherals initialized directly in `main.c/main.h`, rather than separate `gpio.c/spi.c` files), the driver's hardware layer file (`SX1278_hw.c`) needs a small adjustment. Change:

```

#include "gpio.h"
#include "spi.h"

```

to:

```

#include "main.h"

```

7.4 Writing the Receiver Code

In `main.c`, include the driver and add the required includes inside `/* USER CODE BEGIN Includes */`:

```

#include "SX1278.h"
#include <stdio.h>

```

Add the driver instance and buffer variables inside `/* USER CODE BEGIN PV */`:

```

SX1278_hw_t SX1278_hw;
SX1278_t SX1278;
int ret;
char buffer[64];

```

Add a `_write` function to redirect `printf` output over UART2, inside `/* USER CODE BEGIN PFP */`:

```

int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart2, (uint8_t*) ptr, len, 50);
    return len;
}

```

Inside `main()`, after peripheral initialization, configure and initialize the LoRa module (`/* USER CODE BEGIN 2 */`):

```
printf("Boot OK\r\n");

SX1278_hw.dio0.port = LORA_DIO0_GPIO_Port;
SX1278_hw.dio0.pin = LORA_DIO0_Pin;
SX1278_hw.nss.port = LORA_NSS_GPIO_Port;
SX1278_hw.nss.pin = LORA_NSS_Pin;
SX1278_hw.reset.port = LORA_RST_GPIO_Port;
SX1278_hw.reset.pin = LORA_RST_Pin;
SX1278_hw.spi = &hspi3;

SX1278_hw = &SX1278_hw;

SX1278_init(&SX1278, 433000000, SX1278_POWER_10DBM,
    SX1278_LORA_SF_7,
    SX1278_LORA_BW_125KHZ, SX1278_LORA_CR_4_5, SX1278_LORA_CRC_EN
    , 10);

printf("SX1278_init done\r\n");

ret = SX1278_LoRaEntryRx(&SX1278, 64, 2000);
printf("EntryRx ret=%d\r\n", ret);
```

Inside the main `while(1)` loop:

```
ret = SX1278_LoRaRxPacket(&SX1278);
if (ret > 0) {
    SX1278_read(&SX1278, (uint8_t*) buffer, ret);
    printf("Received (%d): %s\r\n", ret, buffer);
}
HAL_Delay(100);
```

7.5 Verifying Communication

Connect to the STM32's serial output (via the ST-LINK virtual COM port) at 38400 baud:

```
screen /dev/cu.usbmodemXXXX 38400
```

With the ESP32 transmitter (configured as sender, Part 3) running nearby, the STM32 should output:

```
Boot OK
SX1278_init done
EntryRx ret=1
Received (20): Hello World!! 101025
Received (20): Hello World!! 101131
Received (20): Hello World!! 101237
```

This confirms successful cross-platform LoRa P2P communication between the ESP32 (transmitter) and STM32F302 (receiver).

7.6 Adding Transmit Capability

The receiver code above only supports receiving. To satisfy the RX/TX learning objective for the STM32 side, transmit capability can be added via a compile-time flag, allowing the same codebase to act as either transmitter or receiver.

Add the following flag inside `/* USER CODE BEGIN Includes */`, alongside the existing includes:

```
#define LORA_TX_MODE 0 // Set to 1 to transmit, 0 to receive
```

Then update the initialization block (`/* USER CODE BEGIN 2 */`) to conditionally skip entry into RX mode when transmitting:

```
#if LORA_TX_MODE
// Entry into TX mode happens per-packet in the loop below,
// since payload length varies each time
#else
ret = SX1278_LoRaEntryRx(&SX1278, 64, 2000);
printf("EntryRx_ret=%d\r\n", ret);
#endif
```

Finally, update the main loop to conditionally transmit instead of receive:

```
#if LORA_TX_MODE
int message_length = sprintf(buffer, "Hello_from_STM32!!_%lu",
    HAL_GetTick());
ret = SX1278_LoRaEntryTx(&SX1278, message_length, 2000);
ret = SX1278_LoRaTxPacket(&SX1278, (uint8_t*) buffer,
    message_length, 2000);
printf("%d_byte_packet_sent:_%s\r\n", message_length, buffer);
HAL_Delay(1000);
#else
ret = SX1278_LoRaRxPacket(&SX1278);
if (ret > 0) {
    SX1278_read(&SX1278, (uint8_t*) buffer, ret);
    printf("Received_(%d):_%s\r\n", ret, buffer);
}
HAL_Delay(100);
#endif
```

Important: `SX1278_LoRaEntryTx()` must be called fresh before every transmission, using the exact length of that specific message. Calling it once outside the loop with a fixed length (e.g. matching only the first message) causes the SPI FIFO address pointer to misalign on subsequent sends, corrupting the received data into a rotated, truncated string.

7.7 Testing STM32 to ESP32 Transmission

To verify the transmit path, set `LORA_TX_MODE` to 1, rebuild, and reflash the STM32. Then flip your ESP32 to receiver mode (Part 3, Configuring TX/RX Mode) and reflash it.

The ESP32 receiver should output:

```
I (964)    RX: 24 byte packet received:[Hello from STM32!! 22990]
I (2054)   RX: 24 byte packet received:[Hello from STM32!! 24084]
I (3144)   RX: 24 byte packet received:[Hello from STM32!! 25178]
```

This confirms the STM32 can both send and receive LoRa packets, completing bidirectional cross-platform communication. Once verified, set `LORA_TX_MODE` back to 0 and re-flash both boards to restore the original receiver/transmitter configuration used throughout the rest of this guide.

8 Part 7: LoRa Parameters and P2P Communication Tests

8.1 Understanding LoRa Communication Parameters

LoRa communication is controlled by three key parameters that determine the trade-off between range, data rate, and robustness. These parameters must be identical on both the transmitter and receiver, otherwise communication will fail.

8.1.1 Spreading Factor (SF)

The spreading factor controls how long each symbol is transmitted. It ranges from SF7 to SF12.

- Higher SF = longer range, slower data rate
- Lower SF = shorter range, faster data rate
- Each increase in SF roughly doubles the time on air and halves the data rate
- SF7 is used for short range, high speed communication
- SF12 is used for maximum range at the cost of very low throughput

8.1.2 Bandwidth (BW)

Bandwidth controls the frequency range used for transmission. Common values are 125 kHz, 250 kHz, and 500 kHz.

- Higher bandwidth = faster data rate, shorter range
- Lower bandwidth = slower data rate, longer range
- 125 kHz (BW7) is the standard default and offers a good balance

8.1.3 Coding Rate (CR)

The coding rate controls how much error correction data is added to each transmission. It ranges from 4/5 to 4/8.

- Higher coding rate = more robust against interference, more overhead

- Lower coding rate = less overhead, less error correction
- CR 4/5 (CR1) is the default and suitable for most applications

Parameter	Value used	Higher value effect	Lower value effect
Spreading Factor	SF7	Longer range, slower	Shorter range, faster
Bandwidth	125 kHz	Faster, shorter range	Slower, longer range
Coding Rate	4/5	More robust, more overhead	Less overhead

The tests below use the default parameters (SF7, 125 kHz, CR 4/5), which prioritise data rate over maximum range. For outdoor long-range applications, increasing the spreading factor to SF10-SF12 would significantly extend the communication range at the cost of throughput.

8.2 EU 433 MHz Power Limits and Duty Cycle

The 433.05-434.79 MHz band used in this guide is regulated under ETSI EN 300 220-2 as a non-specific short range device band, with two key restrictions:

- **Maximum power:** 10 mW (10 dBm) effective radiated power (ERP)
- **Duty cycle:** transmissions are limited to 10% of any given time period (i.e. no more than 6 minutes of transmission per hour)

Both the ESP32 and STM32 implementations in this guide are configured to transmit at 10 dBm to comply with this limit (see Part 3 and Part 6). The example code's 1-second send interval, used throughout testing, is well within the 10% duty cycle allowance given the short packet transmission time. For applications with more frequent transmissions, the duty cycle limit should be calculated based on actual time-on-air per packet.

8.3 Test Setup

Use the parameters above for both tests. Power the transmitter via a phone USB connection (or a USB wall charger) so it can be moved freely without a laptop connection, while keeping the receiver connected to your computer to monitor incoming packets.

8.4 Test 1: ESP32 to ESP32

With one ESP32 flashed as transmitter and the other as receiver (Part 3), move the transmitter away from the receiver and watch the receiver's serial monitor. Each packet includes an incrementing tick counter, so gaps in the sequence indicate lost packets.

In our test, the transmitter was moved approximately 8 metres horizontally to a front door, then down 2 floors (approximately 6 metres vertical) through concrete floors and walls. This gave 0% packet loss over the entire route:

```
I (854)  RX: 19 byte packet received:[Hello World!! 17385]
I (1914) RX: 19 byte packet received:[Hello World!! 17491]
I (2974) RX: 19 byte packet received:[Hello World!! 17597]
```

Packets arrived consistently every 1060 ms, matching the transmitter's 1 second send interval.

8.5 Test 2: ESP32 to STM32F302

Repeat the same test with the ESP32 as transmitter and the STM32F302 as receiver (Part 5). Following the same route, we again observed 0% packet loss:

```
Received (19): Hello World!! 10389
Received (19): Hello World!! 10495
Received (19): Hello World!! 10601
Received (19): Hello World!! 10707
```

This confirms the ported STM32 implementation performs equivalently to the original ESP32 implementation.

8.6 Throughput

Each packet in these tests carried 18-20 bytes of payload, sent once per second. This 18-20 bytes/second figure reflects the 1-second send interval used in the example code, not a property of the LoRa link itself.

The actual physical layer (PHY) bit rate at SF7, 125 kHz bandwidth, and coding rate 4/5 is approximately 5.5 kbps, calculated as:

```
Rb = SF x (4 / (4 + CR)) / (2^SF / BW)
    = 7 x (4/5) / (2^7 / 125000)
    = ~5470 bps
```

Achieving throughput closer to this PHY rate would require removing the artificial send delay and running a back-to-back transmission test, which is left as an exercise for further testing (see below).

8.7 Notes for Further Testing

- These results were gathered indoors over a short distance (15-20m) with significant obstruction (concrete floors/walls) and still saw 0% packet loss, which is typical for LoRa at SF7.
- For longer outdoor range tests, consider increasing the spreading factor (SF10-SF12) at the cost of throughput.
- If you observe packet loss, try comparing results at different spreading factors to see the range/speed trade-off directly.